EXPERIMENT NO: 03	

1. AIM

To find the minimum and maximum numbers from a given list using the Divide & Conquer methodology and compare its performance with the traditional/straightforward approach.

2. THEORY

The Max-Min problem aims to find the maximum and minimum elements in a given array. We can solve this either through a straightforward iterative approach or an optimized Divide & Conquer recursive approach.

2.1 Traditional / Straightforward Approach

In the traditional approach, the array is scanned sequentially. Each element is compared with the current maximum and minimum values and updated accordingly. This requires **(n - 1)** comparisons for finding the maximum and another **(n - 1)** comparisons for finding the minimum, leading to a total of **2(n - 1)** comparisons.

2.2 Divide & Conquer Approach

The Divide & Conquer approach solves the problem in three main stages:

- **Base Case 1 (Single element):** If a[1] is the only element in the sub-array, it is simultaneously both the maximum and minimum.
- **Base Case 2 (Two elements):** If the sub-array contains only two elements, a single comparison between them determines the minimum and maximum.
- **Divide & Combine (More than two elements):** The array is divided at the midpoint mid = ⌊(i + j)/2⌋ into two equal subproblems. Each subproblem is solved recursively, and their results are combined by taking max(max1, max2) and min(min1, min2).

3. ALGORITHMS

Algorithm 1: StraightMaxMin (Traditional Approach)

```
Algorithm StraightMaxMin(a, n, max, min)
// Set max to the maximum and min to the minimum of a[1 : n].
{
    max := min := a[1];
    for i := 2 to n do
    {
        if (a[i] > max) then max := a[i];
        if (a[i] < min) then min := a[i];
```

```
    }
}
```

Algorithm 2: MaxMin (Divide & Conquer Approach)

```
Algorithm MaxMin(i, j, max, min)
// a[1 : n] is a global array. Parameters i and j are integers, 1 <= i <= j <= n.
// Sets max and min to the largest and smallest values in a[i : j].
{
    if (i = j) then
        max := min := a[i]; // Small(P): Single element
    else if (i = j - 1) then // Small(P): Two elements
    {
        if (a[i] < a[j]) then
        {
            max := a[j];
            min := a[i];
        }
        else
        {
            max := a[i];
            min := a[j];
        }
    }
    else
    {
        // Divide P into subproblems
        mid := ⌊(i + j) / 2⌋;

        // Solve subproblems recursively
        MaxMin(i, mid, max, min);
        MaxMin(mid + 1, j, max1, min1);

        // Combine solutions
        if (max < max1) then max := max1;
        if (min > min1) then min := min1;
    }
}
```

4. COMPLEXITY ANALYSIS & MATHEMATICAL DERIVATION

The conventional algorithm takes **2(n - 1)** comparisons in all cases (worst, best, and average). For Divide & Conquer, the recurrence relation is defined as:

```
T(n) = 0                    , if n = 1
T(n) = 1                    , if n = 2
T(n) = 2 T(n/2) + 2        , if n > 2
```

Derivation via Iterative Substitution Method:

Step 1: `T(n) = 2 T(n/2) + 2    --- (Eq. 1)`

Step 2: Substitute n by (n/2) in Eq. 1: $T(n/2) = 2\,T(n/4) + 2$
$\Rightarrow T(n) = 2\,[2\,T(n/4) + 2] + 2 = 4\,T(n/4) + 4 + 2$ --- (Eq. 2)

Step 3: Substitute n by (n/4) in Eq. 1: $T(n/4) = 2\,T(n/8) + 2$
$\Rightarrow T(n) = 4\,[2\,T(n/8) + 2] + 4 + 2 = 8\,T(n/8) + 8 + 4 + 2$
$\Rightarrow T(n) = 2^3\,T(n / 2^3) + 2^3 + 2^2 + 2^1$

Step k-1: After (k - 1) iterations:
$\Rightarrow T(n) = 2^{(k-1)}\,T(n / 2^{(k-1)}) + \Sigma_{(i=1)}^{(k-1)}\,2^i$
$\Rightarrow T(n) = 2^{(k-1)}\,T(n / 2^{(k-1)}) + (2^k - 2)$

Final Substitution: Let $n = 2^k \Rightarrow 2^{(k-1)} = n / 2$
$\Rightarrow T(n) = (n / 2)\,T(2^k / 2^{(k-1)}) + (n - 2)$
$\Rightarrow T(n) = (n / 2)\,T(2) + (n - 2)$
Since T(2) = 1 (2 elements require 1 comparison):
$\Rightarrow T(n) = (n / 2) * 1 + n - 2 = (3n / 2) - 2$

Comparison Summary:

Approach	Number of Comparisons	Time Complexity
Traditional / Straightforward	2(n - 1)	O(n)
Divide & Conquer	(3n / 2) - 2	O(n)

5. PROGRAM CODE (C LANGUAGE)

```
#include <stdio.h>

int max, min;
int a[100];

void maxmin(int i, int j) {
    int max1, min1, mid;

    // Base Case 1: Single element
    if (i == j) {
        max = min = a[i];
    }
    // Base Case 2: Two elements
    else if (i == j - 1) {
        if (a[i] < a[j]) {
            max = a[j];
            min = a[i];
        } else {
            max = a[i];
            min = a[j];
        }
    }
    // Divide & Conquer Step
    else {
        mid = (i + j) / 2;

        // Solve subproblem 1
```

```
        maxmin(i, mid);
        max1 = max;
        min1 = min;

        // Solve subproblem 2
        maxmin(mid + 1, j);

        // Combine solutions
        if (max < max1)
            max = max1;
        if (min > min1)
            min = min1;
    }
}

int main() {
    int i, num;

    printf("\nEnter the total number of numbers : ");
    scanf("%d", &num);

    printf("Enter the numbers : \n");
    for (i = 1; i <= num; i++) {
        scanf("%d", &a[i]);
    }

    max = a[1];
    min = a[1];

    maxmin(1, num);

    printf("\nMinimum element in array : %d\n", min);
    printf("Maximum element in array : %d\n", max);

    return 0;
}
```

6. SAMPLE INPUT & OUTPUT

```
Enter the total number of numbers : 10
Enter the numbers :
1000
800
550
890
999
33
3
34
6
7
```

```
Minimum element in an array : 3
Maximum element in an array : 1000
```

7. CONCLUSION

By implementing the Divide & Conquer approach, the minimum and maximum elements of a list were successfully computed. While both algorithms exhibit linear time complexity O(n), Divide & Conquer reduces the number of element comparisons from 2(n - 1) to [(3n/2) - 2], resulting in approximately a 25% reduction in total comparison operations.